

MODULE 2: INTRODUCTION

Contents

1. Data Flow Diagram (DFD)	1
2. Logical vs Physical DFD	1
3. Control Flow Diagram / Flowchart	2
4. Decision Tree	3
5. Decision Table.....	3
6. Software Project Plan	3
7. Characteristics of a Good SPMP Document	4
8. Function-Oriented vs Object-Oriented Design.....	5
9. Problem Partitioning.....	6

Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graphical representation of data flow within a system, illustrating how input data is processed to generate output data. It helps in understanding how data moves through a system and interacts with different components. **Components of a DFD**

- **External Entities** (Source/Sink): Represent external systems, organizations, or people that interact with the system (e.g., users, external databases).
- **Processes**: Represent activities or functions where data is processed (e.g., calculations, database queries).
- **Data Stores**: Represent storage locations (e.g., databases, files) where data is kept for future use.
- **Data Flows**: Represent the movement of data between processes, data stores, and external entities.
- **DFD Levels**: DFDs are divided into levels to represent the system at varying levels of detail:
 - **Level 0 (Context Diagram)**: This is the highest level, providing a very high-level overview of the entire system. It shows the system as a single process and identifies the primary input and output data flows between the system and external entities.
 - **Level 1**: Breaks down the single process from the Level 0 diagram into more detailed subprocesses. It provides a deeper understanding of the major processes in the system.
 - **Level 2**: Further breaks down the Level 1 processes into more granular subprocesses, showing how each function works and how data moves within smaller units of the system.

Logical vs Physical DFD

Aspect	Logical DFD	Physical DFD
Definition	A Logical DFD focuses on the business functions and processes without considering the actual system architecture.	A Physical DFD focuses on how the system is implemented, including hardware, software, files, and people.

Purpose	To represent the business processes and data flow independent of physical implementation.	To depict how the system will be physically implemented in terms of resources and hardware.
Components	Data Processes, Data Stores, External Entities, Data Flows	Processes, Data Stores, External Entities, Data Flows, Files, Devices, Hardware, and People
Level of Abstraction	Higher level of abstraction, focusing on what the system does (the <i>logic</i> of operations).	Lower level of abstraction, focusing on how the system will be built and how it operates physically.
Focus	Describes what needs to be done in the system, emphasizing system requirements from a functional perspective.	Describes how the system will be implemented, including the physical aspects of the system's environment.
Technology Independent	Does not consider the hardware, software, or physical architecture involved in the system.	Considers actual hardware, software, and other technologies that will be used in the system implementation.
Examples of Components	- Process (e.g., "Process Orders") - Data Store (e.g., "Customer Data") - Data Flow (e.g., "Order Information") - External Entity (e.g., "Customer")	- Process (e.g., "Order Processing System") - Data Store (e.g., "Customer Database") - Physical Data Flow (e.g., "Data Transfer between Database and Application") - Physical Device (e.g., "Server")
When Used	Used in the early stages of system design to understand and model business needs without worrying about implementation.	Used in the later stages of system design to plan how the system will be constructed and the resources required for implementation.
Audience	Mostly intended for business analysts, stakeholders, and users who need to understand the business processes.	Intended for system developers, engineers, and IT staff involved in system implementation.
Example	A logical DFD might show how customer orders are processed in a retail system without specifying the database or software used.	A physical DDF would specify the database (e.g., MySQL), the server (e.g., Linux-based), and the data paths between components.

Control Flow Diagram / Flowchart

A **flowchart** or **control flow diagram** is a graphical representation used to depict the flow of control in a process or system. It visually describes the sequence of steps, decisions, and actions that must be taken to achieve an objective. **Components of a Flowchart**

- **Start/End (Oval):** Indicates the beginning and end points of the process.
- **Process (Rectangle):** Represents a step or action in the process.

- **Decision (Diamond):** Represents a decision point where a yes/no question or a condition is evaluated.
- **Input/Output (Parallelogram):** Denotes input or output data to/from the system.
- **Arrows:** Indicate the flow direction from one step to the next.

Example Flowchart

Decision Tree

A **decision tree** is a tree-like graphical representation used to make decisions based on certain conditions. It is often used in machine learning and decision-making scenarios, helping to visualize and evaluate the consequences of choices. **Components of a Decision Tree**

- **Root Node:** The starting point, representing the main decision or condition.
- **Internal Nodes:** Represent the conditions or tests applied to the data.
- **Branches:** Show the possible outcomes of the conditions.
- **Leaf Nodes:** Represent the final decision or outcome of the decision-making process.

Example of a Decision Tree

Decision Table

A **decision table** is a tabular representation that maps different conditions and the actions that result from them. It is especially useful in situations where multiple conditions can result in different actions. **Components of a Decision Table**

- **Conditions:** Represent the criteria or rules that need to be checked.
- **Actions:** Represent the corresponding actions to be taken based on the conditions.
- **Rules:** The columns that map specific combinations of conditions to their corresponding actions.

Example of a Decision Table

- If the customer is employed **and** has sufficient income (Rule 1), the loan is **approved**.
- For other combinations of employment and income (Rules 2, 3, 4), the loan is **denied**.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Is Customer Employed?	Yes	Yes	No	No
Has Sufficient Income?	Yes	No	Yes	No
Action	Approve	Deny	Deny	Deny

Software Project Plan

A **Software Project Plan (SPP)** outlines the key steps, resources, timeline, and goals for a software development project. It provides a roadmap for how the project will be executed, managed, and delivered. A well-defined project plan ensures that the team stays on track, risks are minimized, and objectives are met within the given constraints. **Key Components of a Software Project Plan:**

1. **Project Scope:** Defines what the project will deliver, including the features, functionalities, and constraints. It outlines the project's boundaries and goals.
2. **Project Objectives:** The specific, measurable, achievable, relevant, and time-bound (SMART) goals the project is aiming to achieve.
3. **Schedule and Milestones:** A detailed timeline of project activities, including the start and end dates, along with key milestones to track progress.
4. **Resource Allocation:** Identifies the human, financial, and technical resources required for the project. This includes team members, budgets, and hardware/software needs.
5. **Risk Management Plan:** Identifies potential risks and outlines strategies for mitigating them. Includes contingency plans for managing risks that materialize.
6. **Communication Plan:** Specifies how project updates, progress reports, and stakeholder communications will be handled throughout the project lifecycle.
7. **Quality Assurance:** Defines the approach for ensuring the final software product meets quality standards, including testing strategies, reviews, and validation processes.
8. **Project Monitoring and Control:** Describes how progress will be tracked and measured, and how deviations from the plan will be addressed to keep the project on track.
9. **Change Management:** Defines the process for handling changes in project scope, requirements, or timelines, ensuring that changes are controlled and documented.

Characteristics of a Good SPMP Document

A **Software Project Management Plan (SPMP)** document outlines how a software project will be executed, monitored, and controlled. It serves as a guide for managing the project's resources, timelines, risks, and deliverables. **Key Characteristics of a Good SPMP Document**

1. **Clarity and Precision:** Clearly define project goals, scope, schedule, budget, and deliverables to avoid misunderstandings.
2. **Comprehensive Planning:** Cover resource allocation, risk management, quality assurance, and communication for all project phases.
3. **Realistic Goals:** Set achievable objectives and milestones based on available time, budget, and resources.
4. **Flexibility:** Allow for adaptability with contingencies for unexpected changes.
5. **Roles and Responsibilities:** Clearly define each team member's role to prevent confusion and ensure accountability.
6. **Milestones and Deliverables:** Specify critical milestones and timelines for deliverables.
7. **Risk Management:** Include a risk management plan with mitigation and contingency strategies.
8. **Resource Management:** Detail the allocation of human, financial, and technical resources.
9. **Performance Monitoring:** Define metrics and checkpoints to track project progress and maintain scope.
10. **Communication Plan:** Set communication guidelines for frequency, format, and participants in updates and meetings.

Function-Oriented vs Object-Oriented Design

Aspect	Function-Oriented Design Approach	Object-Oriented Design Approach
Definition	Focuses on decomposing the system into a set of functions or procedures that perform specific tasks.	Focuses on decomposing the system into a collection of interacting objects that encapsulate data and behaviour.
Main Concept	Organized around functions (procedures) and algorithms that transform inputs into outputs.	Organized around objects that represent real-world entities and have attributes (data) and methods (behaviour).
Decomposition	Breaks down the system into functions or modules, which are defined by their operations.	Breaks down the system into objects and classes, each representing a real-world entity with both data and behavior.
Modularity	Modules correspond to functions, and interaction happens through function calls.	Modules correspond to classes or objects, and interaction happens through object methods (messages).
Data Handling	Data is passed between functions, and often there is no central place to store data.	Data is encapsulated within objects, and access to it is controlled via methods (encapsulation).
Design Focus	Focuses on processes and how the data flows through the system.	Focuses on data (in the form of objects) and the interactions between these objects.
State Management	Functions typically operate on data, but they don't retain state information across calls.	Objects maintain their own state through attributes and can change their state through methods.
Reuse	Reusability is achieved by breaking down tasks into small, reusable functions.	Reusability is achieved through inheritance, polymorphism, and composition of objects.
Flexibility & Extensibility	Harder to adapt as changes in one function can require changes in other functions due to tightly coupled logic.	More flexible and extensible due to inheritance, encapsulation, and polymorphism, which allows changes in objects without affecting others.
Example	Example: A payroll system designed around functions like calculateSalary() and deductTaxes().	Example: A payroll system designed with classes like Employee, Salary, and Tax where each object interacts and manages its own data.
Design Representation	Typically represented using data flow diagrams (DFDs) and flowcharts that show how functions interact.	Typically represented using UML diagrams (Class Diagrams, Sequence Diagrams, etc.) that model objects and their interactions.
Primary Goal	To transform inputs into desired outputs using functional decomposition.	To model the problem domain by creating objects that interact with each other to fulfill system requirements.
Coupling	Tends to have higher coupling because functions depend on data being passed between them.	Tends to have lower coupling due to encapsulation, where objects manage their own data.
Abstraction	Focuses on the abstraction of processes and algorithms.	Focuses on the abstraction of real-world entities as objects.

Maintenance	Maintenance can be challenging due to tight coupling between functions and data dependencies.	Easier to maintain due to modularity, encapsulation, and clear separation of responsibilities.
Real-World Mapping	Limited real-world mapping since it revolves around functional procedures.	Direct real-world mapping by modelling entities as objects that interact with each other.

Problem Partitioning

Problem Partitioning is a design and analysis technique used in software engineering and systems development. It involves breaking down a complex problem into smaller, more manageable sub-problems or components. This approach can enhance understanding, simplify the design process, and facilitate easier implementation and maintenance.

To make complex problems more manageable, understandable, and solvable by focusing on smaller, discrete components.

Benefits

1. Simplifies problem analysis and design
2. Enhances clarity and understanding
3. Facilitates parallel development
4. Improves maintainability
5. Supports better testing and debugging

Types of Partitioning

1. **Functional Partitioning:** Dividing based on different functions or operations.
2. **Data Partitioning:** Dividing based on different data sets or structures.
3. **Task Partitioning:** Dividing based on different tasks or responsibilities.
4. **Layered Partitioning:** Dividing based on different layers of abstraction (e.g., presentation layer, business logic layer, data access layer).

Implementation of Problem Partitioning

1. **Identify the Overall Problem:** Understand the complete problem that needs to be solved.
2. **Break Down the Problem:** Divide the main problem into smaller, manageable sub-problems or modules.
3. **Analyse Sub-Problems:** Assess each sub-problem to understand its requirements, constraints, and how it interacts with other components.
4. **Define Interfaces:** Establish clear interfaces for communication between the different sub-problems or modules.
5. **Implement and Test:** Develop each partition independently, followed by integration testing to ensure they work together as expected.